

Esta segunda parte de la práctica consiste en transformar el traductor de la sesión previa. El traductor previo seguía un esquema conocido como traducción directa, es decir, la conversión e impresión de la entrada analizada se hace sobre la marcha al mismo tiempo que se genera el árbol sintáctico.

En la nueva propuesta aplicaremos un esquema de traducción algo más elaborado, aunque menos sofisticado que el uso de Árboles Sintácticos Abstractos, que se verán más adelante.

Para ésta ocasión emplearemos también una entrada de expresiones en notación infija, pero la salida será en notación prefija.

Trabajo a realizar:

Para ésta segunda aproximación se pide únicamente la traducción de expresiones aritméticas y de asignaciones a variables de una notación infija a la en el Lenguaje C a notación prefija que serán evaluadas por un intérprete de Lisp.

1. Leed el enunciado completo antes de comenzar.
2. Utilizad el fichero proporcionado **calc7b.y** y renombradlo a **calc7.y**
3. Adaptad la gramática y la semántica para que traduzca las expresiones introducidas a notación prefija de forma que pueda ser evaluado por el intérprete online: https://rextester.com/l/common_lisp_online_compiler.
4. Evaluad los resultados usando el intérprete online. Para ello podéis:
 - a. Editar un fichero (por ejemplo, **prueba.txt**) con una serie de expresiones
 - b. Ejecutar **cat prueba.txt | ./calc7**
 - c. Copiar la salida y pegarla en la ventana del intérprete
 - d. Con F8 se compila y ejecuta.

Entrega:

Subid vuestro fichero **calc7.y**.

Incluid en la cabecera del fichero dos líneas de comentario iniciales. La primera con vuestros nombres y número de grupo. Y la segunda con los correos electrónicos (separados con un espacio).

Entregad también un documento llamado **calc7.pdf** con una breve explicación del trabajo realizado.

Después de hacer la entrega, descargad vuestro fichero y comprobad que funciona correctamente y que cumple con las indicaciones.

Sobre la notación prefija

Una expresión en notación infija como $1+2*3$ se representa en notación prefija como: $(+ 1 (* 2 3))$. La conversión debe tener en cuenta la precedencia y asociatividad que definen las convenciones. En la notación prefija, los paréntesis sirven para establecer la relación de los operadores con sus respectivos operandos. Al mismo tiempo, determinan con qué precedencia y asociatividad se evalúan los operadores.

Una expresión como $(* (+ 1 2) 3)$ se entiende que hace referencia a la equivalente en notación infija: $(1+2)*3$ en la que se ha forzado una precedencia alternativa a la convencional.

Una sentencia de asignación entre variables y expresiones como $a = b*7 + c$; debe traducirse como $(= a (+ (* b 7) c))$.

Especificaciones

Proponemos una secuencia de pasos para abordar esta práctica:

1. Estudiad detenidamente el ejercicio resuelto de conversión entre notaciones con **código diferido**. Comprobad las estructuras de datos y las funciones que se usan.
2. Estudiad el código **calc7.y** proporcionado:
 - a. La estructura de datos para los *Token* **struct s_attr**, dado que abandonamos el uso de **%union**.
 - b. También se suprime la directiva **%type** que asocia un campo determinado (atributo) de la **union** a cada uno de los *No Terminales* y *Token* que intervienen en la gramática y que deben transmitir un valor hacia arriba en el árbol sintáctico.
 - c. Qué funciones coinciden con las incluidas en el ejercicio resuelto.
 - d. Cómo opera la función **yylex()**.
3. No modifiquéis la función **yylex()**.
4. Incluid las acciones semánticas adecuadas en las producciones para que la salida pase de infija a prefija.
 - a. El abandono del uso de **%type** que asocia un campo determinado de la **%union** a los *No Terminales* conlleva que habrá que operar con **\$\$cadena** (atributo explícito) en vez de con **\$\$** (atributo implícito).¹
 - b. En este caso, no se debe imprimir nada en la salida estándar en los nodos inferiores. La salida debe ser añadida en el campo **\$\$cadena** para transmitirla hacia arriba. La impresión de la traducción debe hacerse en el axioma.
 - c. La entrada $1+2*3$ debería traducirse como $(+ 1 (* 2 3))$.
 - d. Prestad atención a la precedencia y asociatividad definidas en la cabecera del fichero **calc7.y** .
5. Abordad la traducción de las *Variables*, tanto en la parte de componentes de una expresión, como receptoras de una asignación.
6. Prestad atención al diseño jerárquico de la gramática. Debería haber una *Sentencia* que deriva en *Expresion* por un lado y en *Asignacion* por otro. La traducción debería generarse a nivel de *Sentencia*. Para separar sentencias usaremos saltos de línea.
7. Incluid una nueva sentencia para imprimir valores. En Lenguaje C debería ser **printf("%d", <expresion>)** pero nuestro Analizador Lexicográfico no está adaptado para reconocer *Palabras Reservadas* (como **printf**), ni tratar la cadena de formato. Por ello proponemos una simplificación. La entrada **@ <expresion>** debe traducirse como **(print <expresion>)**.

¹ No recomendamos usar el sistema de atributos implícitos por los problemas que causa si se olvida aplicar a un no terminal y por coherencia con los exámenes que piden atributos explícitos.